

# AirSim + YOLOv8

## 无人机自动追踪算法原理与实现

本文档说明双无人机自动追踪系统的算法原理和工程实现。系统以 Tracker 无人机前视相机为输入，通过 YOLOv8 锁定 Target 无人机，再使用图像平面 Kalman 预测器估计未来短时位置，最后由视觉伺服控制器驱动 Tracker 保持目标位于画面中心。

### 1. 系统目标与架构

- Tracker 是主控无人机，负责采集图像和执行追踪控制。
- Target 是被追踪无人机，是检测器唯一需要识别的 drone 类别。
- 闭环目标是 Target 飞到哪里，Tracker 就追到哪里，同时 Target 不脱离视线。
- 控制输入来自相机画面中的检测框，而不是直接使用 AirSim 真值位置。

```
Tracker camera frame
-> YOLOv8 detector
-> ImagePlaneKalmanPredictor
-> VisualServoController
-> AirSim body velocity / yaw / gimbal command
-> next camera frame
```

### 2. YOLOv8 目标锁定

YOLOv8 对 Tracker 相机图像进行推理，输出目标无人机边界框 Detection。边界框使用 xyxy 像素格式，并携带置信度 confidence 和类别 class\_id。检测框中心和尺寸是后续预测与控制的唯一视觉状态。

```
cx = (x1 + x2) / 2
cy = (y1 + y2) / 2
w  = x2 - x1
h  = y2 - y1
```

#### 重新采集数据的原因

旧数据可能包含非无人机目标、目标过小或标注策略不一致。当前方案要求目标就是 Target 无人机，采集时让目标更大、更靠近画面中心，并尽量露出机架和螺旋桨，使模型学习真正的无人机外观特征。

### 3. 图像平面 Kalman 运动预测

预测器使用常速度模型，不直接预测三维真值坐标，而是在图像平面预测检测框。这样能直接补偿采集、推理、控制命令和机体响应带来的延迟，也能在短时丢检时继续给控制器一个合理目标。

| 状态量    | 含义              |
|--------|-----------------|
| cx, cy | 目标框中心像素坐标       |
| w, h   | 目标框宽高，反映目标距离和尺度 |
| vx, vy | 目标中心在图像平面的速度    |
| vw, vh | 目标框尺寸变化速度       |

```
state = [cx, cy, w, h, vx, vy, vw, vh]^T
measurement = [cx, cy, w, h]^T
```

```

x(k|k-1) = F(dt) x(k-1|k-1)
x_future = F(horizon_s) x(k|k)

```

## 状态转移模型

预测器采用常速度模型。每个控制周期根据真实时间差  $dt$  更新位置和尺寸，速度项保持不变。这等价于假设目标在很短时间内近似匀速运动。对 0.2 秒量级的前馈预测，这个假设比复杂神经网络更可靠，也更容易调参。

```

F(dt) =
[1 0 0 0 dt 0 0 0]
[0 1 0 0 0 dt 0 0]
[0 0 1 0 0 0 dt 0]
[0 0 0 1 0 0 0 dt]
[0 0 0 0 1 0 0 0]
[0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 1]

```

## Kalman 更新步骤

YOLO 给出新检测框时，预测器用观测  $z=[cx,cy,w,h]$  修正状态。如果检测框有轻微抖动，Kalman 增益会在历史运动趋势和当前观测之间做平衡。

```

y = z - H x_pred
S = H P_pred H^T + R
K = P_pred H^T S^-1
x = x_pred + K y
P = (I - K H) P_pred

```

其中  $Q$  由 `process_noise` 控制，表示目标运动模型的不确定性； $R$  由 `measurement_noise` 控制，表示 YOLO 观测的不确定性。 $Q$  越大，预测器越愿意相信目标可能突然改变速度； $R$  越大，预测器越不容易被单帧检测噪声拉偏。

## 关键参数

| 参数                                | 默认值  | 作用                        |
|-----------------------------------|------|---------------------------|
| <code>horizon_s</code>            | 0.2  | 预测未来 0.2 秒的位置，用于补偿视觉和控制延迟 |
| <code>max_prediction_gap_s</code> | 0.6  | 丢检后继续使用预测的最长时间            |
| <code>process_noise</code>        | 80.0 | 越大越允许目标突然加速或转向            |
| <code>measurement_noise</code>    | 25.0 | 越大越不容易被 YOLO 单帧抖动带偏       |

## 短时丢检策略

如果 YOLO 暂时没有检测到 Target，且距离最近一次真实检测没有超过 `max_prediction_gap_s`，预测器继续输出预测框。超过阈值后返回 `None`，原有搜索扫描逻辑接管。

```

if raw_detection is not None:
    predict(dt)
    update(raw_detection)
    return predict(horizon_s)

if raw_detection is None and age <= max_prediction_gap_s:
    predict(dt)
    return predict(horizon_s)

return None

```

## 4. 视觉伺服控制

控制器追踪的是预测框中心。设图像宽高为 W,H，预测中心为 (cx\_hat, cy\_hat)，归一化中心误差为：

```
ex = (cx_hat - W / 2) / (W / 2)
ey = (cy_hat - H / 2) / (H / 2)
```

- Yaw 控制：yaw\_rate = clip(k\_yaw \* ex, -max\_yaw\_rate, max\_yaw\_rate)。
- 垂直控制：vz = clip(k\_z \* ey, -max\_vertical\_speed, max\_vertical\_speed)。
- 距离控制：使用目标框宽度，而不是强依赖深度图。框太小则前进，框太大则后退。
- 云台辅助：云台 yaw 快速拉回水平中心，pitch 积分默认关闭以避免长时间漂移。

### 控制律细节

deadzone:

```
if abs(ex) < center_deadzone: ex = 0
if abs(ey) < center_deadzone: ey = 0
```

```
yaw_rate = clip(k_yaw * ex, -max_yaw_rate_deg_s, max_yaw_rate_deg_s)
vz        = clip(k_z * ey, -max_vertical_m_s, max_vertical_m_s)
```

```
box_width_norm = predicted_w / image_width
size_error = desired_box_width_norm - box_width_norm
vx = clip(k_size * size_error, -max_reverse_m_s, max_forward_m_s)
```

这里 vx 使用框宽而不是深度图作为主要距离信号，是因为无人机机架和螺旋桨细节容易让深度图局部不稳定。框宽是检测器直接输出的视觉量，和“画面里目标大小是否合适”这个追踪目标更一致。

### 丢失恢复层级

| 阶段   | 触发条件                        | 动作               |
|------|-----------------------------|------------------|
| 短时丢检 | age <= max_prediction_gap_s | 继续用 Kalman 预测框控制 |
| 预测超时 | age > max_prediction_gap_s  | 预测器返回 None       |
| 搜索扫描 | 控制器没有可用目标                   | 云台或机体按最后一次目标方向扫描 |

## 5. 运行时实现

```
raw_detection = detector.detect(frame.rgb)
prediction = predictor.step(
    detection=raw_detection,
    timestamp_s=loop_start,
    image_width=image_width,
    image_height=image_height,
)
command = controller.command(
    prediction.detection,
    frame.depth,
    image_width,
    image_height,
    lost_time,
)
```

PredictionResult.detection 的类型仍然是 Detection，因此控制器接口不需要改变。这一点很重要：预测器是插在检测器和控制器之间的增强层，而不是推翻原有控制结构。

### 工程文件映射

| 文件                                             | 职责                                              |
|------------------------------------------------|-------------------------------------------------|
| src/ drone_tracker/ detector. py               | 封装 YOLOv8, 输出 Detection                         |
| src/ drone_tracker/ predictor. py              | 实现 ImagePlaneKalmanPredictor 和 PredictionResult |
| src/ drone_tracker/ controller. py             | 根据预测框中心误差和框宽生成控制命令                              |
| src/ drone_tracker/ metrics. py                | 写出 tracking CSV 和 summary JSON                  |
| scripts/ run_tracking. py                      | 主闭环, 串联 AirSim、YOLO、预测器、控制器和日志                  |
| config/ tracking_ config. json                 | 启用预测的生产配置                                       |
| config/ tracking_ config_ no_ prediction. json | 关闭预测的 A/ B 对照配置                                 |

主循环顺序

```
1. move_target(...)          # AirSim ■■■ Target
2. frame = get_scene_and_depth(...) # Tracker ■■■■
3. raw = detector.detect(frame.rgb) # YOLO ■■■■
4. pred = predictor.step(raw, t)   # Kalman ■■■■■■■■
5. cmd = controller.command(pred.detection, ...)
6. set_camera_gimbal(...)         # ■■■■■■
7. move_body_velocity(...)        # ■■ Tracker ■■■■
8. write TrackingSample           # ■■■■■■■■■■
```

6. 日志指标

CSV 在原有追踪字段基础上增加预测字段：

- prediction\_used: 当前控制是否使用预测结果。
- predicted\_center\_x / predicted\_center\_y: 预测框中心。
- raw\_center\_x / raw\_center\_y: YOLO 原始检测中心。
- prediction\_age\_s: 距离最近一次真实检测的时间。

summary 指标包括 visible\_rate、lost\_count、max\_lost\_duration\_s、center\_error\_mean\_px、center\_error\_p95\_px 和 prediction\_used\_rate。

指标解释

| 指标                   | 解释             | 理想结果          |
|----------------------|----------------|---------------|
| visible_rate         | 控制周期中目标可用的比例   | 越接近 1 越好      |
| lost_count           | 连续丢失段数量        | 0             |
| max_lost_duration_s  | 最长连续丢失时间       | 0 或尽可能小       |
| center_error_p95_px  | 95% 情况下的中心误差上界 | 低于基线 67.71 px |
| prediction_used_rate | 预测器参与控制的比例     | 用于理解预测器实际作用   |

7. 验证与验收

```
python -m compileall src scripts tests
python tests\test_predictor.py
```

```
python scripts\run_tracking.py --config config\tracking_config.json ^
--weights D:/sim/runs/drone_tracker/detect/drone_centered_yolov8s_v2/weights/best.pt ^
--seconds 90
```

```
python scripts\run_tracking.py --config config\tracking_config_no_prediction.json ^
--weights D:/sim/runs/drone_tracker/detect/drone_centered_yolov8s_v2/weights/best.pt ^
--seconds 90
```

| 验收项                 | 目标              |
|---------------------|-----------------|
| visible_rate        | $\geq 0.98$     |
| lost_count          | 0               |
| center_error_p95_px | $\leq 67.71$ px |

A/B 对照很重要：只看启用预测的一次结果，无法判断收益来自预测器还是目标轨迹更简单。因此需要用同一权重、同一目标策略、同一运行时长分别跑 prediction.enabled=true 和 false。

## 8. 调参建议

- 追踪慢半拍：将 horizon\_s 从 0.2 增加到 0.25。
- 突然转向过冲：将 horizon\_s 降到 0.15。
- 检测框抖动明显：增大 measurement\_noise。
- 预测跟不上快速运动：增大 process\_noise。
- 丢检后预测漂移：降低 max\_prediction\_gap\_s。

## 9. 局限与后续方向

当前预测器只在图像平面工作，适合短时延迟补偿和短时丢检恢复，不适合长时间无观测预测。后续可以加入相机内参和框宽估距，形成 3D EKF；也可以用 AirSim 日志训练 GRU/LSTM，处理更复杂的机动模式。

### 典型问题定位

| 现象        | 可能原因                    | 优先处理                            |
|-----------|-------------------------|---------------------------------|
| 目标总是偏左或偏右 | yaw 控制符号或云台方向不匹配        | 检查 ex 与 yaw_rate 的符号            |
| 目标上下漂移    | pitch 积分漂移或 k_z 偏小      | 关闭 pitch 积分，增大 k_z              |
| 目标忽大忽小    | k_size 过大或目标框抖动         | 降低 k_size 或增大 measurement_noise |
| 短时丢检后乱追   | max_prediction_gap_s 过长 | 降低到 0.3 到 0.5                   |
| 转弯时过冲     | horizon_s 过大            | 降低到 0.15                        |

## 10. 结论

自动追踪的核心是检测、预测、控制和恢复机制组成的闭环。YOLOv8 负责锁定目标无人机，Kalman 预测器根据历史检测框估计未来短时位置，视觉伺服控制器负责把目标拉回画面中心。这套结构实时、可解释、可调试，适合当前 AirSim 双无人机追踪任务。